Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа №3
по дисциплине «Анализ Алгоритмов»

Тема Поиск в словаре

Студент Платонова М. И.

Группа ИУ7-51Б

Преподаватель Волкова Л. Л.

Москва — 2025 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Аналитическая часть **4**

1.1 Алгоритмы поиска элементов в массиве 4

1.1.1 Поиск полным перебором 4

1.1.2 Бинарный поиск 4

2 Конструкторская часть **6**

2.1 Описание структуры ПО 6

2.2 Описание алгоритмов 6

2.3 Алгоритм взаимодействия с пользователем 6

3 Технологическая часть **8**

3.1 Требования к программному обеспечению 8

3.2 Средства реализации 8

3.3 Структура программы 8

3.4 Описание работы программы 8

3.5 Реализация алгоритмов 9

3.6 Функциональные тесты 11

4 Исследовательская часть **12**

4.1 Влияние позиции элемента на количество сравнений в алгоритмах 12

4.2 Поиск полным перебором 12

4.3 Бинарный поиск 13

ЗАКЛЮЧЕНИЕ **15**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **16**

ВВЕДЕНИЕ

В данной лабораторной работе рассматриваются два метода поиска элементов в массиве: поиск полным перебором и бинарный поиск. Линейный поиск выполняется путём последовательного сравнения каждого элемента массива с искомым значением, пока оно не будет обнаружено или не завершится обход всех элементов. Бинарный поиск ускоряет этот процесс, сокращая область поиска в два раза на каждом шаге, начиная с проверки среднего элемента оставшегося диапазона, но требует заранее отсортированного массива.

Алгоритмы поиска имеют важное значение в программировании, так как массивы применяются во многих сферах, включая компьютерную графику, анализ данных и машинное обучение. Повышение эффективности поиска позволяет значительно уменьшить время обработки данных, улучшая производительность программ и облегчая работу разработчика.

Целью данной лабораторной работы является исследование двух алгоритмов поиска элемента в массиве: бинарного и линейного (полным перебором).

Для достижения этой цели необходимо выполнить следующие задачи:

– рассмотреть и реализовать алгоритм линейного поиска элемента в массиве;

– рассмотреть и реализовать алгоритм бинарного поиска элемента в массиве;

– провести исследование зависимости количества сравнений от позиции искомого элемента в массиве;

– подготовить графики, отображающие результаты сравнительного анализа;

– разработать набор тестов для оценки работоспособности реализованных алгоритмов;

– сформулировать и представить результаты в отчёте о проведённой лабораторной работе.

1 Аналитическая часть

В этом разделе рассматриваются два метода поиска элементов в массиве: поиск полным перебором и бинарный поиск [1].

1.1 Алгоритмы поиска элементов в массиве

1.1.1 Поиск полным перебором

Поиск полным перебором – один из простейших алгоритмов для нахождения элемента в массиве. Он предполагает последовательное сравнение каждого элемента массива с искомым значением, начиная с первого элемента и до тех пор, пока либо не будет найдено совпадение, либо не будут проверены все элементы массива.

Пусть задан массив длины n. Сложность алгоритма для такого массива оценивается как $O(n)$. Линейный поиск не требует дополнительных структур данных или сортировки массива и занимает минимальное количество памяти. В каждой итерации цикла алгоритма выполняются операции индексации и сравнения. Для данного алгоритма выделяют один лучший и два худших случая по количеству операций:

$$f = \begin{cases} 1, & \text{если элемент находится на первой позиции} \\ n, & \text{иначе, если элемент не найден или найден на последнем сравнении} \end{cases} \tag{1.1}$$

Пусть на старте алгоритм поиска выполняется k_0 операций, и при каждом сравнении требуется ещё k_1 операций (где k_0 и k_1 – константы). Тогда количество операций можно выразить следующим образом:

- В лучшем случае, когда элемент находится на первой позиции: $k_0 + k_1$
- Если элемент найден на второй позиции: $k_0 + 2 \cdot k_1$
- В случае, если элемент найден на последней позиции: $k_0 + n \cdot k_1$
- Если элемент не найден совсем $k_0 + n \cdot k_1$

1.1.2 Бинарный поиск

Бинарный поиск является более оптимизированным методом для поиска в отсортированном массиве, поскольку он сокращает количество элементов для проверки на каждой итерации, что ускоряет процесс.

Алгоритм бинарного поиска делит массив на две части на каждом шаге, проверяя, совпадает ли искомый элемент со значением в середине текущего диапазона. Если значение не совпадает, то в зависимости от результата сравнения выбирается либо левая, либо правая половина массива для следующей проверки. Этот процесс продолжается, пока не будет найден элемент или пока массив не будет исчерпан.

Пусть массив имеет длину n. Сложность алгоритма бинарного поиска оценивается как $O(\log n)$, поскольку количество проверок уменьшается на каждом шаге вдвое. Для бинарного поиска, как и для линейного, выделяются один лучший и два худших случая:

$$f = \begin{cases} 1, & \text{если элемент находится в середине массива} \\ \log_2(n), & \text{иначе, если элемент отсутствует или найден на последней проверке} \end{cases} \tag{1.2}$$

Если элемент находится в середине массива, его можно найти за одну операцию. Если же элемент отсутствует или находится в краю массива, нам нужно будет пройти через все уровни бинарного деления, чтобы проверить последний элемент. Бинарный поиск требует, чтобы массив был отсортирован перед началом поиска, но не требует дополнительных структур данных. Это делает его эффективным для больших объёмов данных, где линейный поиск становится непрактичным.

Вывод

В данном разделе рассмотрены алгоритмы полным перебором и бинарного поиска. Линейный поиск удобен для неупорядоченных массивов, но его сложность $O(\log n)$ ограничивает эффективность при больших объёмах данных. Бинарный поиск, обладающий сложностью $O(\log_2 n)$, работает гораздо быстрее на отсортированных массивах, что делает его подходящим для работы с большими данными, несмотря на необходимость предварительной сортировки.

2 Конструкторская часть

В данном разделе представлены описания алгоритмов поиска элемента в массиве, а также структуры программного обеспечения и оценки трудоёмкости для каждого метода.

2.1 Описание структуры ПО

На вход программного продукта передан массив arr, состоящий из целых чисел, и случайное значение, которое необходимо найти. Программа позволяет пользователю выбрать один из двух алгоритмов поиска: поиск полным перебором или бинарный поиск. В случае бинарного поиска массив предварительно сортируется.

2.2 Описание алгоритмов

В программном продукте реализованы два алгоритма поиска:

– Линейный поиск, выполняющий последовательный перебор элементов массива для нахождения заданного значения.

– Бинарный поиск, который применяется к предварительно отсортированному массиву и позволяет эффективно находить элемент.

Для сортировки массива перед выполнением бинарного поиска используется алгоритм сортировки слиянием. Алгоритм сортировки слиянием делит массив на части, которые сортируются и объединяются, обеспечивая упорядоченность элементов массива.

2.3 Алгоритм взаимодействия с пользователем

Программа взаимодействует с пользователем через консоль:

– Пользователь вводит номер индивидуальной задачи, который влияет на параметры генерации массива;

– Генерируется массив заданной длины из случайных целых чисел;

– Программа выполняет линейный поиск случайного элемента и выводит его позицию и время выполнения;

– Массив сортируется с помощью алгоритма сортировки слиянием;

– Выполняется бинарный поиск первого элемента отсортированного массива, и выводятся его позиция и время выполнения.

На рисунках 2.1 и 2.2 представлены схемы реализованных алгоритмов линейного и бинарного поиска.

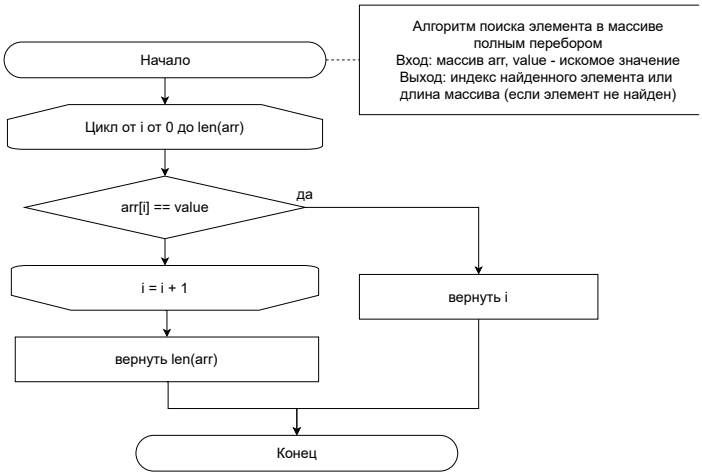

Рисунок 2.1 — Алгоритм поиска элемента в массиве полным перебором

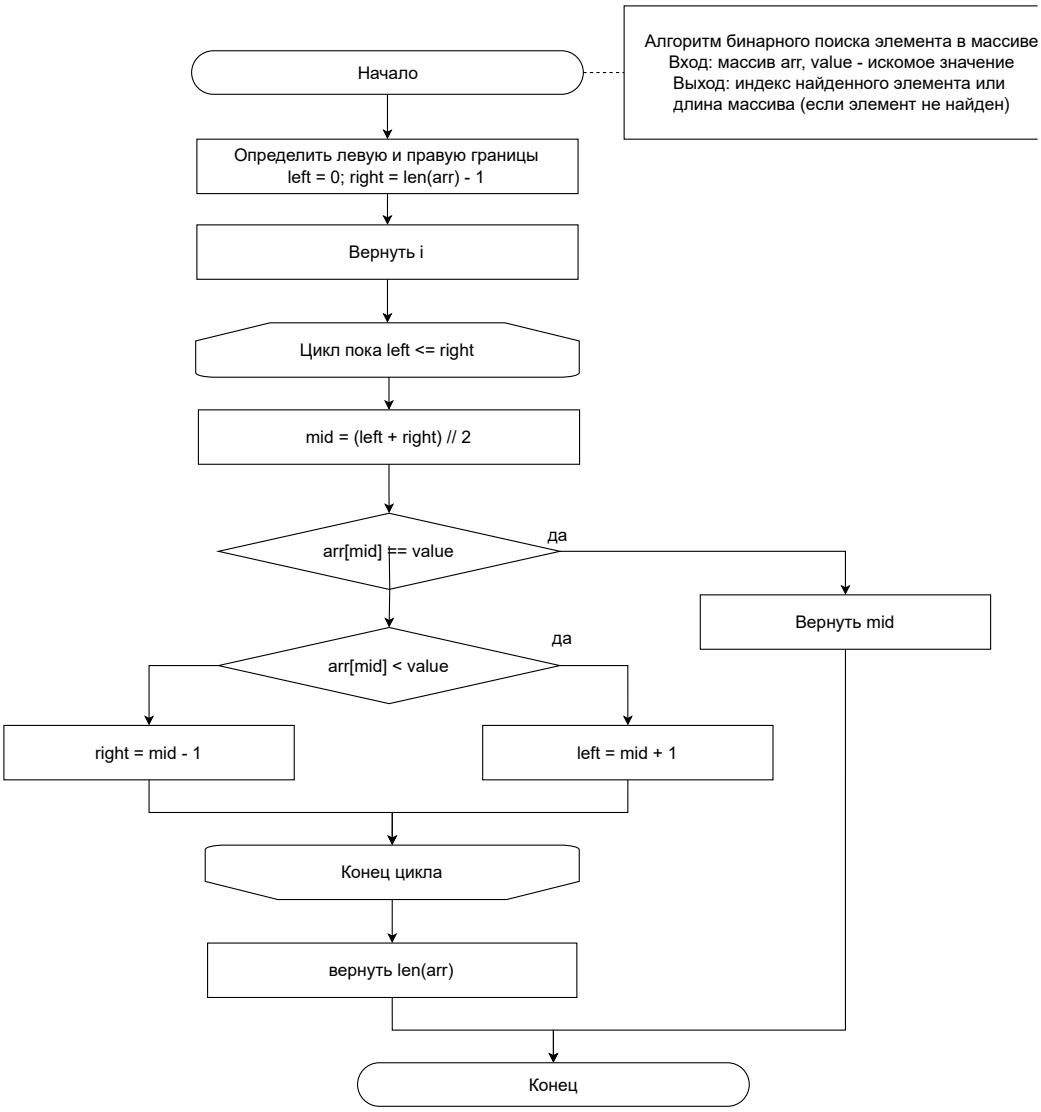

Рисунок 2.2 — Алгоритм бинарного поиска элемента в массиве

3 Технологическая часть

В данном разделе приведены данные о выбранном языке программирования, реализации алгоритмов поиска и тестировании каждого алгоритма.

3.1 Требования к программному обеспечению

– Входные данные: массив `arr`, состоящий из целых чисел, и значение, которое необходимо найти в этом массиве.

– Выходные данные: индекс найденного элемента в массиве или, если элемент не найден, длина массива (что указывает на отсутствие значения).

3.2 Средства реализации

Для реализации алгоритмов поиска, проведения замеров времени и обработки данных в данной работе используется язык программирования `Python` [3]. `Python` предоставляет все необходимые инструменты для работы с массивами и измерения времени выполнения.

Для разработки алгоритмов используются следующие типы и структуры данных:

– Одномерный массив целых чисел `arr`, в котором выполняется поиск.

– Переменные для хранения текущих индексов и границ поиска.

Замеры времени выполняются с помощью функции *time* [4].

3.3 Структура программы

Программа состоит из нескольких модулей:

– `algs.py` – файл, содержащий реализацию всех алгоритмов;

– `compare.py` – модуль для сравнения времени выполнения алгоритмов поиска.

3.4 Описание работы программы

Программа взаимодействует с пользователем через консоль:

– Пользователь вводит номер индивидуальной задачи, что влияет на генерацию параметров массива.

– Генерируется массив случайных чисел заданной длины и диапазона.

– Выполняется линейный поиск случайного значения в массиве и выводится его позиция и время выполнения.

– Массив сортируется с использованием сортировки слиянием для подготовки к бинарному поиску.

– Выполняется бинарный поиск первого элемента отсортированного массива, выводятся его позиция и время выполнения.

3.5 Реализация алгоритмов

В листинге 3.1 представлена реализация функции `calculate_length`, которая вычисляет длину на основе заданного числа X.

```
def calculate_length(X):
    if X // 8 + (X >> 2) % 10 == 0:
        return X % 1000
    else:
        return ((X >> 2) % 10 * (X % 10) + (X >> 1) % 10)
```

Листинг 3.1 — Реализация функции расчёта длины на основе номера задачи

В листинге 3.2 представлена реализация функции `normal_search`, которая осуществляет линейный поиск элемента в массиве.

```
def normal_search(arr, value):
    for i in range(len(arr)):
        if arr[i] == value:
            return i
    return len(arr)
```

Листинг 3.2 — Реализация линейного поиска элемента в массиве

В листинге 3.3 представлена реализация функции `binary_search`, которая выполняет бинарный поиск элемента в отсортированном массиве.

```
def binary_search(arr, value):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == value:
            return mid
        elif arr[mid] < value:
            left = mid + 1
        else:
            right = mid - 1
    return len(arr)
```

Листинг 3.3 — Реализация бинарного поиска элемента в отсортированном массиве

В листинге 3.4 представлена реализация функции `merge`, которая объединяет две части массива во время выполнения сортировки слиянием.

```
def merge(arr, left, mid, right):
    n1 = mid - left + 1
    n2 = right - mid
    left_part = arr[left:left + n1]
    right_part = arr[mid + 1:mid + 1 + n2]

    i = j = 0
    result = []
    while i < n1 and j < n2:
        if left_part[i] <= right_part[j]:
            result.append(left_part[i])
            i += 1
        else:
            result.append(right_part[j])
            j += 1

    result.extend(left_part[i:])
    result.extend(right_part[j:])

    for k in range(n1 + n2):
        arr[left + k] = result[k]
```

Листинг 3.4 — Реализация функции слияния для сортировки

В листинге 3.5 представлена реализация функции `merge_sort`, которая сортирует массив с помощью алгоритма сортировки слиянием.

```
def merge_sort(arr, left, right):
    if left < right:
        mid = (left + right) // 2
        merge_sort(arr, left, mid)
        merge_sort(arr, mid + 1, right)
        merge(arr, left, mid, right)
```

Листинг 3.5 — Реализация сортировки слиянием

3.6 Функциональные тесты

В таблицах 3.1 и 3.2 приведены функциональные тесты для алгоритмов бинарного поиска и полного перебора элементов в массиве. Все тесты выполнены успешно, фактические результаты совпадают с ожидаемыми.

Размер массива	Массив	Элемент поиска	Бинарный поиск	Ожидаемый результат
0	[]	0	не найден, кол-во сравн. 0	не найден, кол-во сравн. 0
1	[0]	0	индекс 0, кол-во сравн. 1	индекс 0, кол-во сравн. 1
1	[0]	1	не найден, кол-во сравн. 1	не найден, кол-во сравн. 1
2	[0, 1]	0	индекс 0, кол-во сравн. 1	индекс 0, кол-во сравн. 1
2	[0, 1]	1	индекс 1, кол-во сравн. 2	индекс 1, кол-во сравн. 2
4	[1, 2, 3, 4]	3	индекс 2, кол-во сравн. 2	индекс 2, кол-во сравн. 2
8	[1, 3, 5, 7, 9, 11, 13, 15]	9	индекс 4, кол-во сравн. 3	индекс 4, кол-во сравн. 3
8	[1, 3, 5, 7, 9, 11, 13, 15]	16	не найден, кол-во сравн. 4	не найден, кол-во сравн. 4

Таблица 3.1 — Функциональные тесты для бинарного поиска элемента в массиве

Размер массива	Массив	Элемент поиска	Полный перебор	Ожидаемый результат
0	[]	0	не найден, кол-во сравн. 0	не найден, кол-во сравн. 0
1	[0]	0	индекс 0, кол-во сравн. 1	индекс 0, кол-во сравн. 1
1	[0]	1	не найден, кол-во сравн. 1	не найден, кол-во сравн. 1
2	[0, 1]	0	индекс 0, кол-во сравн. 1	индекс 0, кол-во сравн. 1
2	[0, 1]	1	индекс 1, кол-во сравн. 2	индекс 1, кол-во сравн. 2
4	[1, 2, 3, 4]	3	индекс 2, кол-во сравн. 3	индекс 2, кол-во сравн. 3
8	[1, 3, 5, 7, 9, 11, 13, 15]	9	индекс 4, кол-во сравн. 5	индекс 4, кол-во сравн. 5
8	[1, 3, 5, 7, 9, 11, 13, 15]	16	не найден, кол-во сравн. 8	не найден, кол-во сравн. 8

Таблица 3.2 — Функциональные тесты для полного перебора элемента в массиве

Вывод

Результаты тестирования подтверждают, что оба алгоритма корректно выполняют поиск: бинарный поиск эффективен для отсортированных массивов, а полный перебор гарантирует нахождение элемента независимо от порядка массива. Все тесты успешно завершены, результаты совпадают с ожидаемыми.

4 Исследовательская часть

В данном разделе представлены результаты исследования эффективности работы алгоритмов по времени выполнения, основанной на критерии временных затрат.

4.1 Влияние позиции элемента на количество сравнений в алгоритмах

Для тестирования алгоритмов использовались данные, представленные в виде словарей, где ключи – это строки, а значения – соответствующие хэши. Объём массива вычислялся по следующей формуле:

$$\text{length} = \begin{cases} X\%1000, & \text{если } X \equiv 0 \mod 8 \\ (X >> 2)\%10 \cdot (X\%10) + (X >> 1)\%10, & \text{иначе} \end{cases} \tag{4.1}$$

где X – номер индивидуального задания.

Для проведения замеров выбран массив длиной 1000 элементов, заполненный случайными хэшами.

4.2 Поиск полным перебором

На рисунке 4.1 представлена гистограмма для линейного поиска.

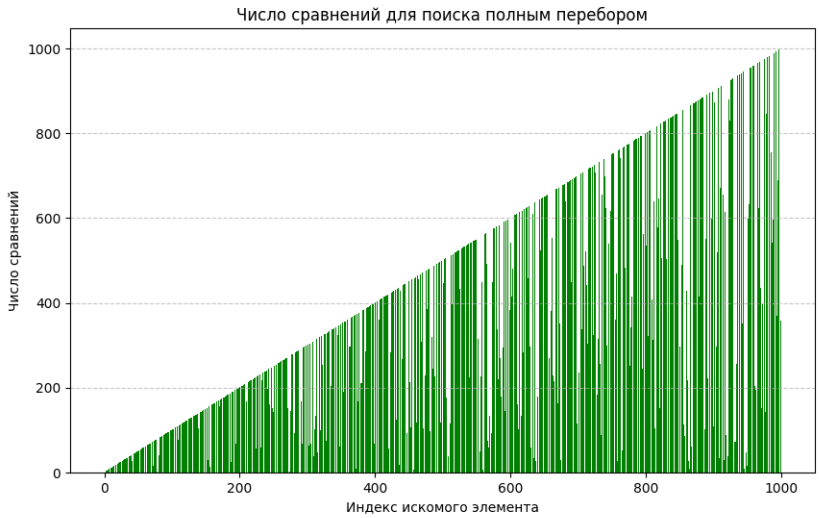

Рисунок 4.1 — Зависимость количества сравнений от позиции элемента в массиве при линейном поиске.

Линейный поиск – это алгоритм, при котором каждый элемент массива проверяется последовательно до тех пор, пока не будет найден нужный элемент или не завершится обход всего массива. Этот метод имеет сложность $(O(n)$, так как в худшем случае требуется пройти весь массив.

Выполненные замеры демонстрируют, что количество сравнений в линейном поиске зависит от позиции элемента: чем ближе элемент к началу массива, тем меньше сравнений требуется для его нахождения. В случае отсутствия искомого элемента линейный поиск выполняет наибольшее количество сравнений, равное длине массива.

4.3 Бинарный поиск

На рисунке 4.2 представлена гистограмма для бинарного поиска.

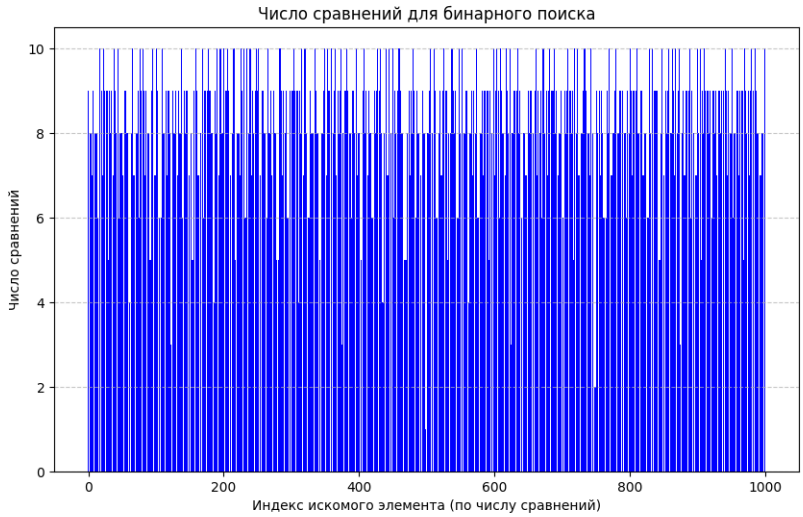

Рисунок 4.2 — Зависимость количества сравнений от позиции элемента в массиве при бинарном поиске.

На рисунке 4.3 представлена гистограмма, отображающая результаты для бинарного поиска.

Рисунок 4.3 — Результаты для бинарного алгоритма.

Бинарный поиск требует предварительной сортировки массива и использует метод деления пополам для поиска элемента. Сложность этого алгоритма равна $O(\log n)$, что позволяет значительно сократить количество сравнений по сравнению с линейным поиском. Однако для выполнения бинарного поиска массив должен быть отсортированным, что может добавить к общей сложности.

При выполнении бинарного поиска для большинства элементов требуется от 8 до 10 сравнений, что делает его гораздо более быстрым по сравнению с полным перебором, где может потребоваться гораздо больше шагов для нахождения элемента.

Вывод

Линейный поиск требует большее количество сравнений по сравнению с бинарным поиском, особенно при увеличении размера массива. В случаях, когда массив уже отсортирован, бинарный поиск является более эффективным методом. Линейный поиск остаётся удобным для неупорядоченных массивов, но при больших объёмах данных бинарный поиск позволяет достичь более высокой скорости за счёт уменьшения количества сравнений.

ЗАКЛЮЧЕНИЕ

Цель работы достигнута. В ходе выполнения данной лабораторной работы решены следующие задачи:

– рассмотрен и реализован алгоритм линейного поиска элемента в массиве;

– рассмотрен и реализован алгоритм бинарного поиска элемента в массиве;

– проведено исследование зависимости количества сравнений от позиции искомого элемента в массиве, что позволило выявить особенности работы каждого алгоритма;

– подготовлены графики, демонстрирующие результаты сравнительного анализа эффективности алгоритмов;

– разработан набор тестов для оценки правильности и производительности реализованных алгоритмов по критериям временных затрат и количества сравнений;

– результаты исследования представлены в виде отчёта о выполненной лабораторной работе.

Исследования показали, что эффективность поисковых алгоритмов зависит от позиции элемента. Бинарный поиск требует меньше сравнений, чем линейный.

При линейном поиске число сравнений растёт с позицией элемента, что делает его неэффективным для больших массивов. Бинарный поиск, наоборот, увеличивает трудоёмкость логарифмически, что сокращает количество сравнений.

Хотя бинарный поиск обычно демонстрирует лучшую эффективность по сравнению с линейным, его скорость может уменьшаться, если элемент расположен в начале массива. Также необходимо учитывать, что его эффективность зависит от используемого метода сортировки массива.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Сборник задач по теории алгоритмов: учеб.-метод. пособие / В. М. Котов [и др.]. – Минск: БГУ, 2017. – 183 с. – ISBN 978-985-566-412-4.

2. Ничушкина, Т. Н., Гуренко, В. В. Разработка алгоритмов простейших программ: Электронное учебное издание. – М.: МГТУ имени Н. Э. Баумана, 2014. – 47 с., ил.

3. Python Software Foundation. Официальная документация по Python [Электронный ресурс]. – Режим доступа: `https://www.python.org/doc/` (дата обращения: 31.10.2024).

4. Python Software Foundation. Модуль time – Доступ к времени и его преобразования [Электронный ресурс]. – Режим доступа: `https://docs.python.org/3/library/time.html` (дата обращения: 31.10.2024).